HDR Implementation: From Theory to Pipeline

A Practical Guide to Mobile Computational Photography | Chapter 13

```
class HDRProcessor:
    HDRI:
```



STANDARD SENSOR CAPTURE (8-BIT)

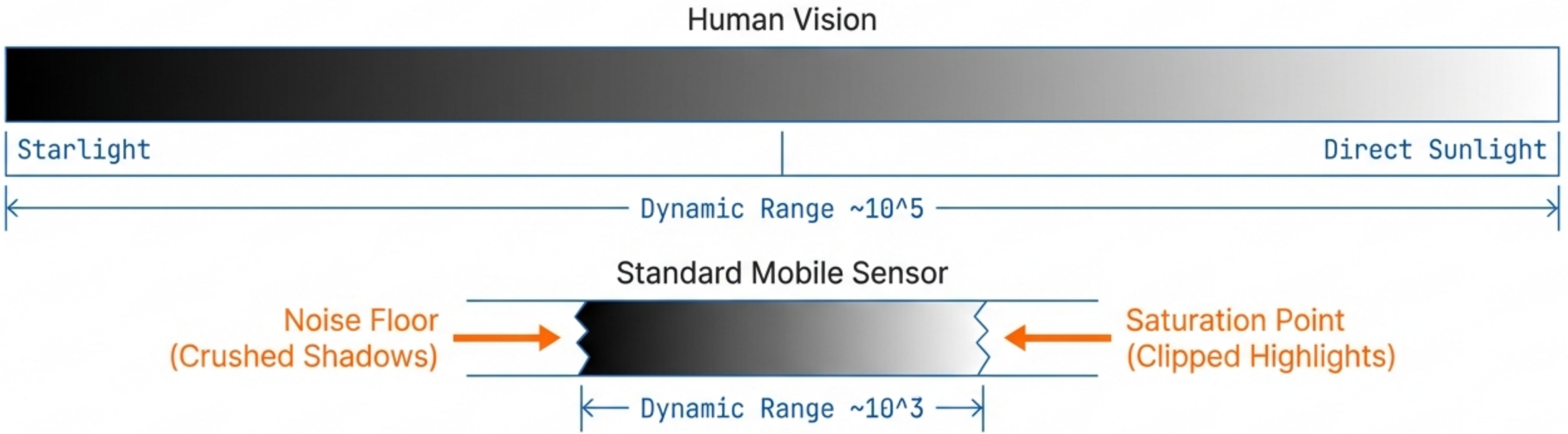


HDR PROCESSED (32-BIT)

Based on Mobile Computational Photography by Jianfeng Ren

NotebookLM

Bridging Physics and Perception

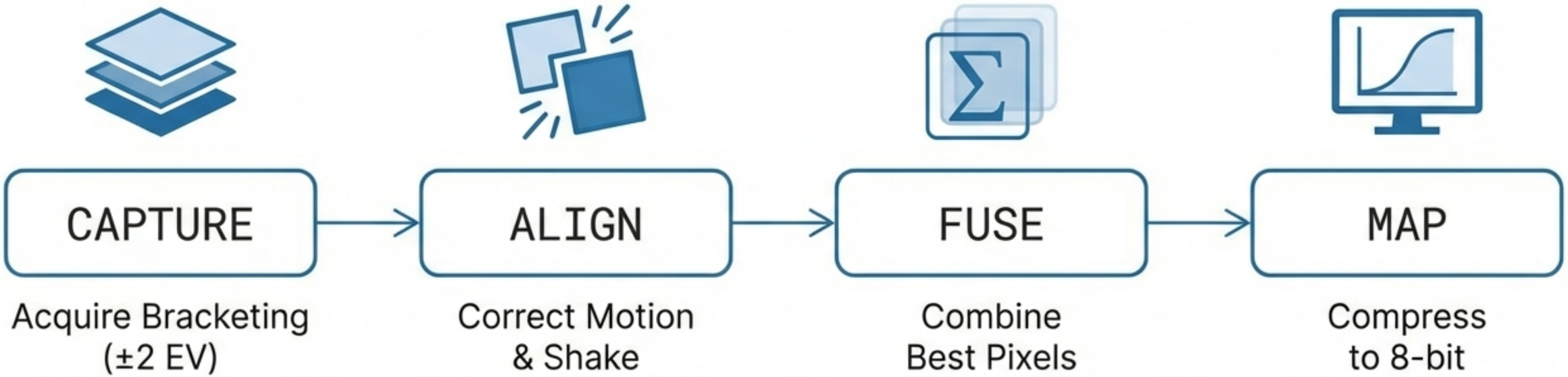


The Engineering Objective

- Simulate multi-frame exposure capture (`Bracketing`).
- Implement feature-based image alignment (`Registration`).
- Execute intelligent image fusion (`Weighted Average`).
- Apply tone mapping for display (`LDR Conversion`).

NotebookLM

The Solution Strategy

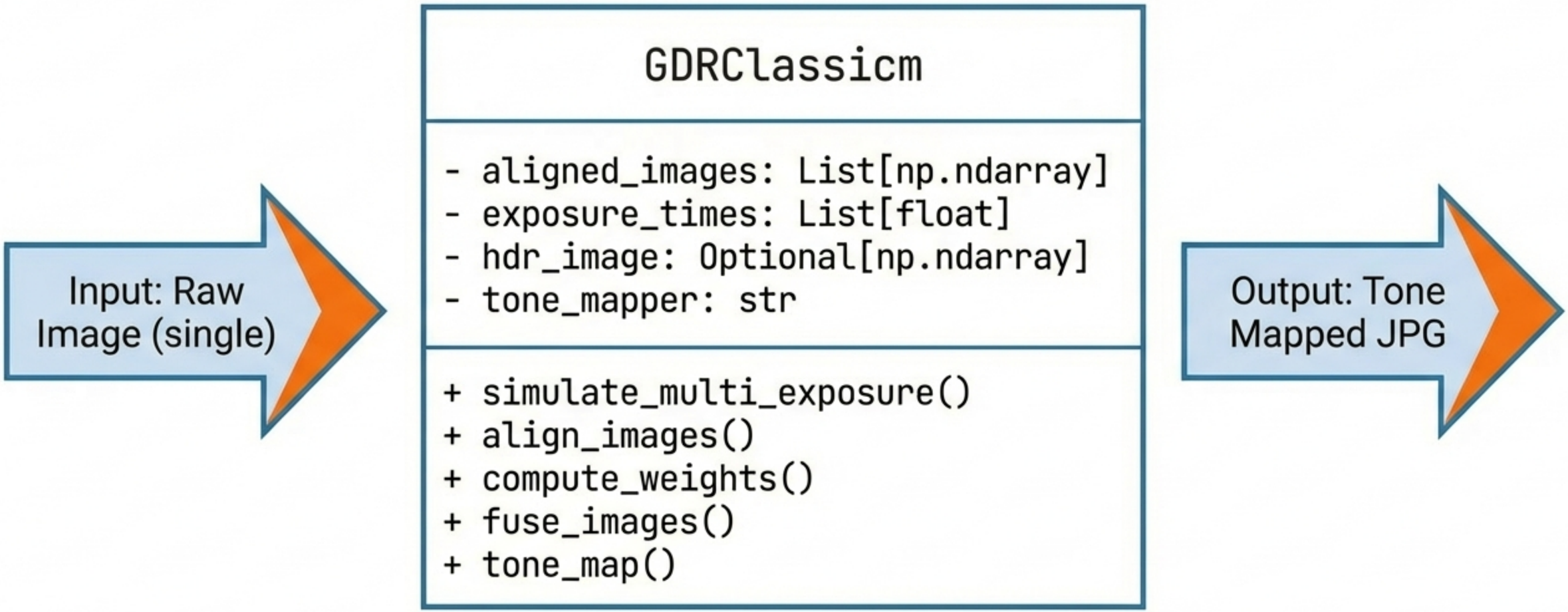


Key Insight

We trade temporal resolution (time to capture multiple frames) for radiometric resolution (higher dynamic range).

NotebookLM

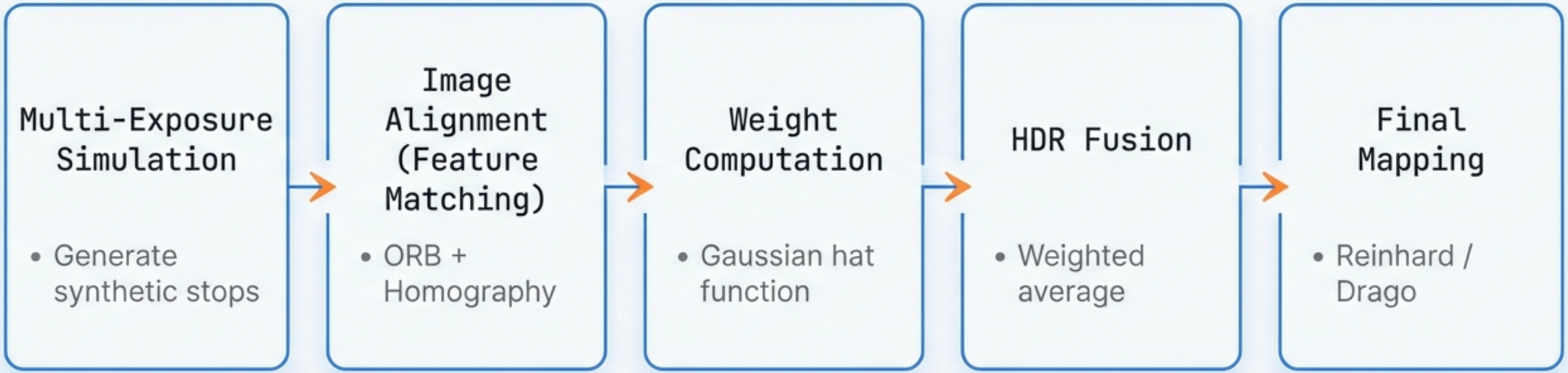
System Architecture



A modular, object-oriented design enabling swappable alignment and mapping algorithms.

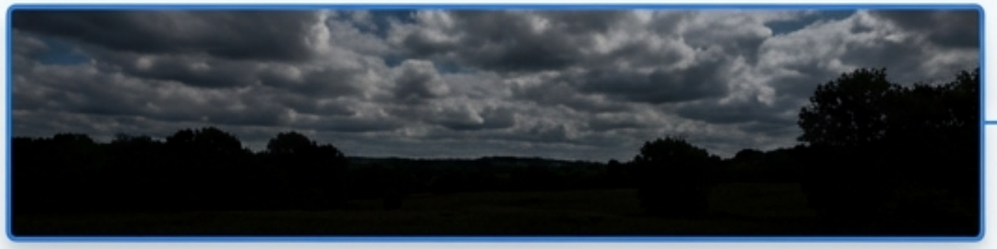
NotebookLM

The 5-Stage Processing Pipeline



NotebookLM

Step 1: Multi-Frame Simulation



0.25x Exposure



1.0x Exposure



4.0x Exposure

```python
def simulate_exposure(image, ratio):
    # Linear scaling of pixel values
    return np.clip(image * ratio, 0, 255)
```

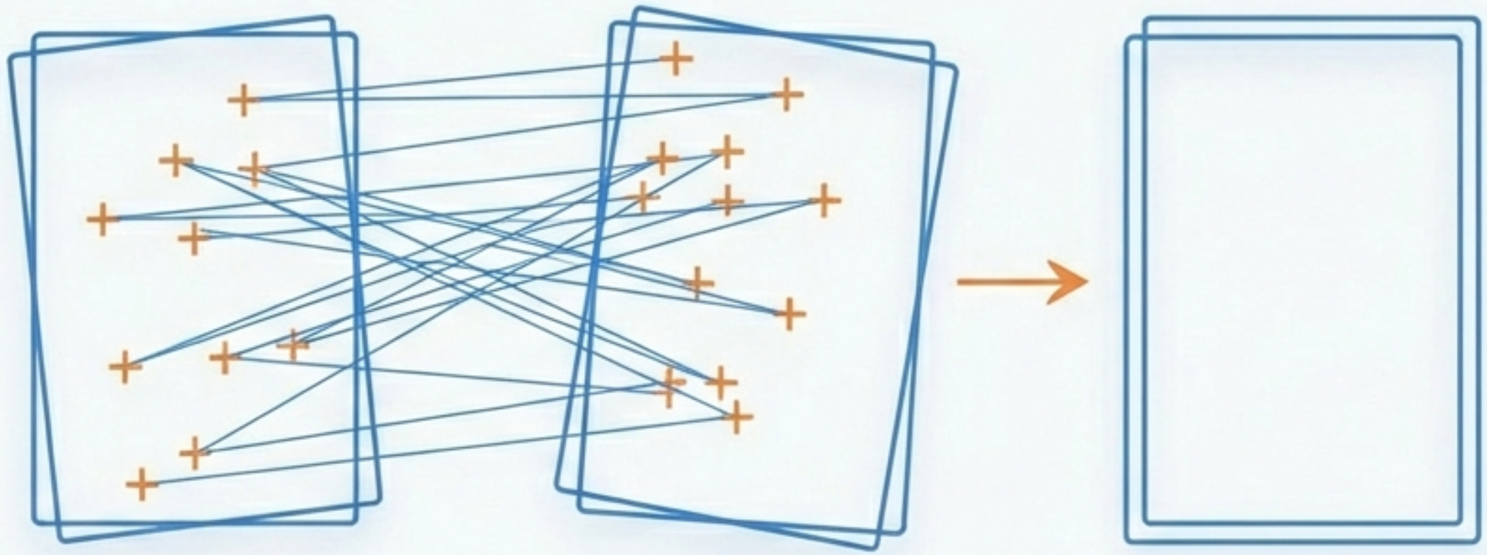
In a real camera, this is achieved by changing shutter speed. In this simulation, we multiply pixel values by exposure ratios to mimic hardware bracketing.

`Standard Ratios: [0.25, 0.5, 1.0, 2.0, 4.0]`

NotebookLM

Step 2: Image Alignment

Feature Matching



FLANN Matching

Homography Transformation

Technical Stack

Detector: ORB (Oriented FAST and Rotated BRIEF)

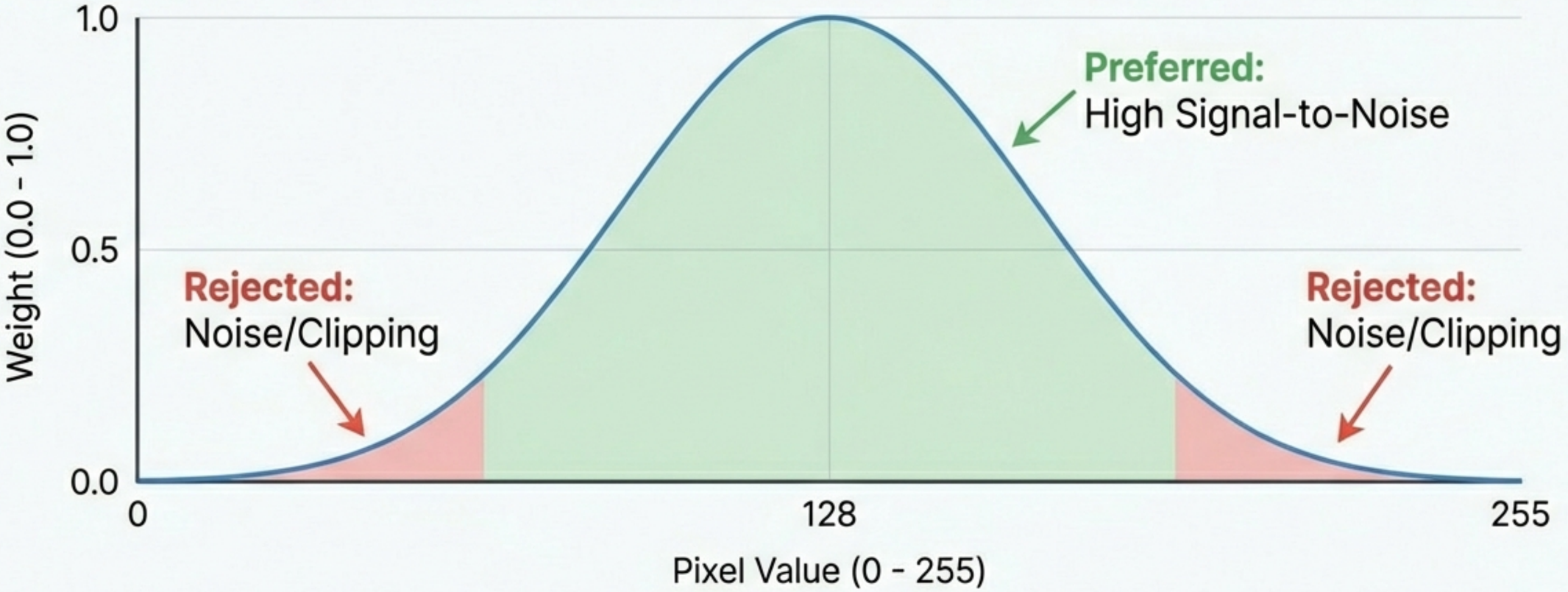
Matcher: FLANN (Fast Library for Approximate Nearest Neighbors)

Transform: Homography Matrix Estimation

Reference: Middle Frame (Index 2)

NotebookLM

Step 3: Weight Computation



Logic: We trust pixels in the middle of the dynamic range. We mistrust pixels that are pitch black (noise) or pure white (saturation).

NotebookLM

Step 4: HDR Fusion

$$\mathrm{HDR}(p) = \frac{\left(\sum w_i(p) * \left(\frac{I_i(p)}{t_i}\right)\right)}{\sum w_i(p)}$$

Weight
(Quality Confidence)

Pixel Value

Exposure Time
(Normalization)

Data Type Transformation

Input: List of 8-bit Integers (0-255)

Fusion Process

Output: Single 32-bit Float (0.0 - >1.0)

NotebookLM

Step 5: Tone Mapping

The Goal: Compress 32-bit range into 8-bit display without losing detail.



Linear Mapping



Reinhard Operator

Preserves local contrast.
Physically based.



Drago Operator

Adaptive logarithmic.
Good for detail preservation.

NotebookLM

Code Example: Running the Pipeline

```python
import cv2
from hdr_example import HDRProcessor

# 1. Load Source Image
image = cv2.imread('sample.jpg')

# 2. Initialize Engine
processor = HDRProcessor()

# 3. Execute Pipeline (Simulation -> Align -> Fuse -> Map)
results = processor.process_hdr_pipeline(image)

# 4. Retrieve Outputs
hdr_32bit = results['hdr_image']     # For scientific use
final_jpg = results['tone_mapped']   # For display display
```

Encapsulates all 5 stages in one call.

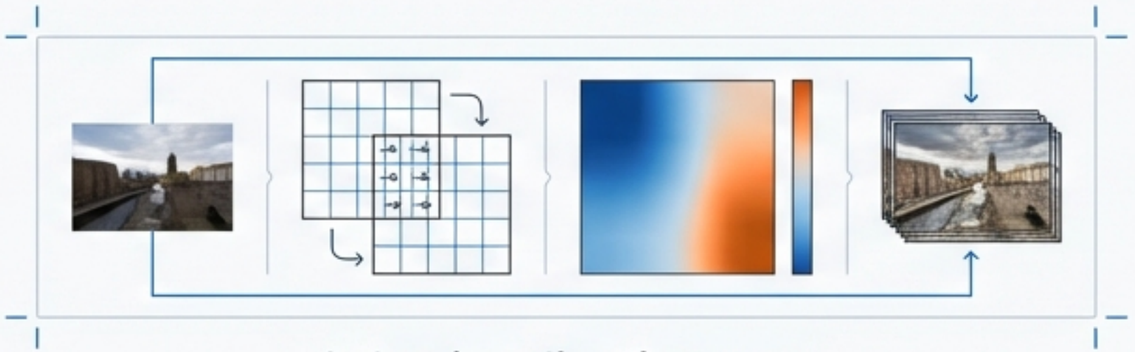
NotebookLM

Advanced Configuration & Output

```
$ python hdr_example.py --input
  photo.jpg --tone-mapping drago
  --exposures 0.25 1.0 4.0
```

Customizable flags

- `--exposures`: Define custom bracket ratios (e.g., ±3 stops)
- `--tone-mapping`: Switch between ‘reinhard’ and ‘drago’
- `--debug`: Save intermediate visualization strips



hdr_visualization.png

NotebookLM

Future Enhancements: AI Integration



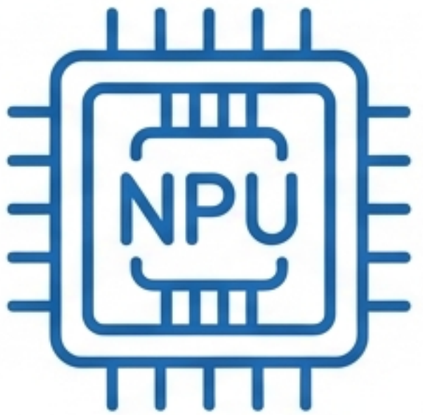
Neural Fusion

Replace weighted average math with CNNs to intelligently weigh pixels and reduce noise.



Motion Deghosting

Deep learning models to detect complex object motion (**people/cars**) that homography misses.



NPU Acceleration

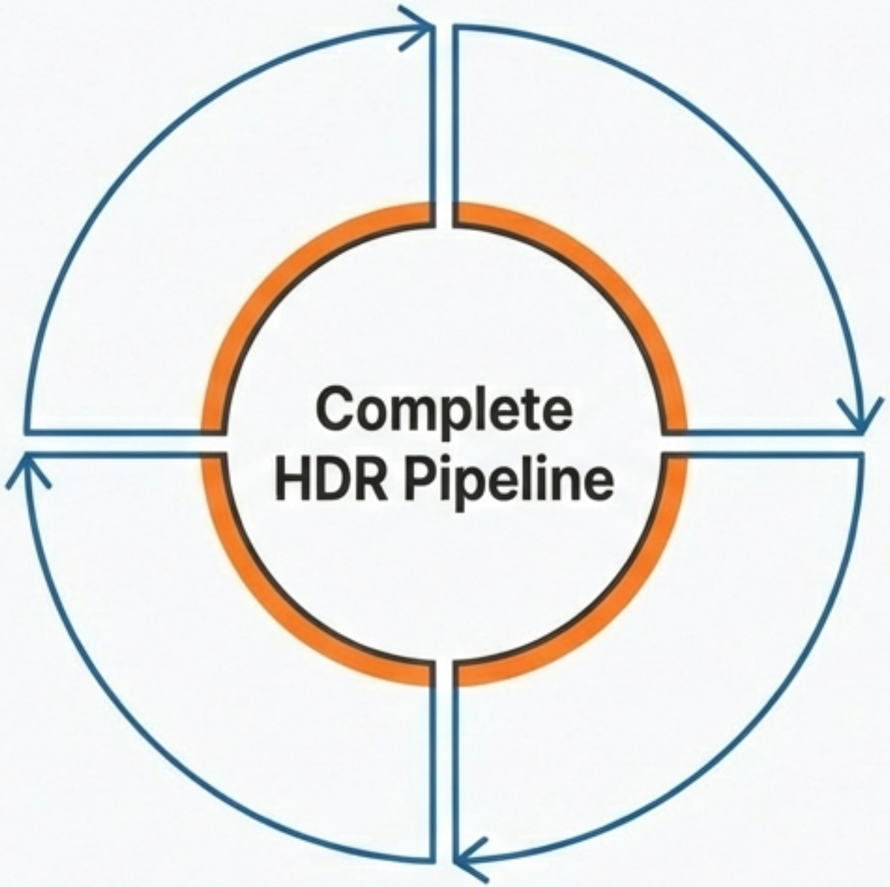
Offload matrix operations to the Neural Processing Unit for real-time 4K processing.

NotebookLM

Summary & Takeaways

Simulation
Modeled hardware bracketing.

Alignment
Fixed temporal offsets.



Mapping
Compressed for display.

Fusion
Merged radiometric data.

https://github.com/JeffRen1977/hdr

Next Up
Chapter 14:
Multi-Camera Solutions

NotebookLM